

The primary objective of BeeWare is to enable the development of native user interfaces that are rich in nature, and are easier to develop and deploy across various types of devices. The goal of the BeeWare project is the development of tools to enable the following:

- Tools to package a Python project in such a manner that it can be executed on various types of devices.
- Providing libraries to access the native capabilities and widgets of these devices.
- Providing tools that facilitate the development, debugging and analysis of these projects.

The primary advantage of using BeeWare is that it uses the native widgets and native functionality in contrast to the theme based approach.

BeeWare is a collection of tools and libraries that work together to build cross-platform native GUI apps in Python. It includes the following:

- **Toga:** A widget toolkit that is supported across various platforms.
- **Briefcase:** A tool that enables the packaging of Python projects.
- **Rubicon ObjC:** A library for working with Objective C libraries from Python code. It is useful in iOS and MacOS.
- **Rubicon Java:** A library for working with Java from Python code.

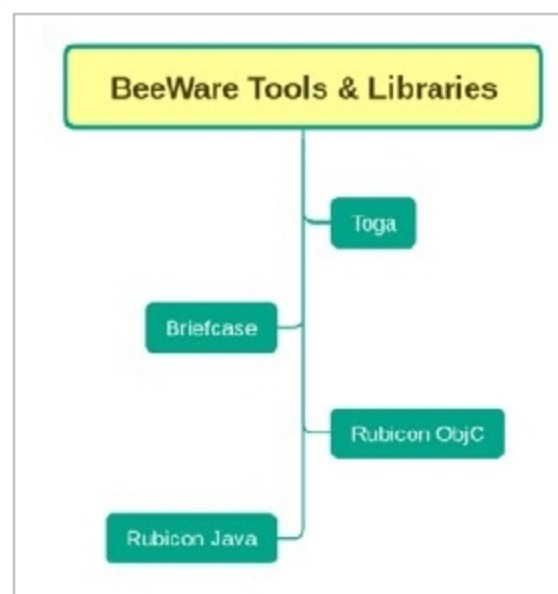


Figure 7: BeeWare tools and libraries

These tools may be used individually or as a suite.

The first step is to install Briefcase, which can be done with the following command:

```
$ python -m pip install briefcase
```

After the successful installation of Briefcase, the first app can be created with the following command:

```
$ briefcase new
```

Briefcase will prompt you to enter the following details:

- Formal name: The default value can be accepted (Hello World)
- App name: Default value can be accepted (helloworld)
- Bundle
- Project name
- Description
- Author
- Author's email
- URL: The landing page of your application
- Licence
- GUI framework: Here, the default option is Toga, which is BeeWare's GUI toolkit, as explained earlier.

Based on the supplied values, the skeleton of the project is created, which will look similar to the one shown in Figure 8.

```

beeware-tutorial/
  beeware-venv/
  ...
  helloworld/
    LICENSE
    README.rst
    pyproject.toml
    src/
      helloworld/
        resources/
          helloworld.icns
          helloworld.ico
          helloworld.png
        __init__.py
        __main__.py
        app.py
  
```

Figure 8: BeeWare project skeleton

You can run this app in the developer mode with the following commands:

```

$ cd helloworld
(beeware-venv) $ briefcase dev

[helloworld] Installing
dependencies...
...
[helloworld] Starting in dev mode...
  
```

You will see the output shown in Figure 9.

Tutorials for customising and deploying apps are available in the official documentation at <https://docs.beeware.org/en/latest/index.html>.

Kivy vs BeeWare

So far in this article, readers have been introduced to two Python based tools to build cross-platform apps — Kivy and BeeWare. Now let's compare them.

- The most important difference between Kivy and BeeWare is the manner in which the widgets/controls are rendered. Kivy provides its own style in rendering widgets and hence they don't look native. However, across the platforms, the applications look consistent. On the other hand, BeeWare uses the native user interface and hence the widgets/controls look native on the respective platforms in which they are running.

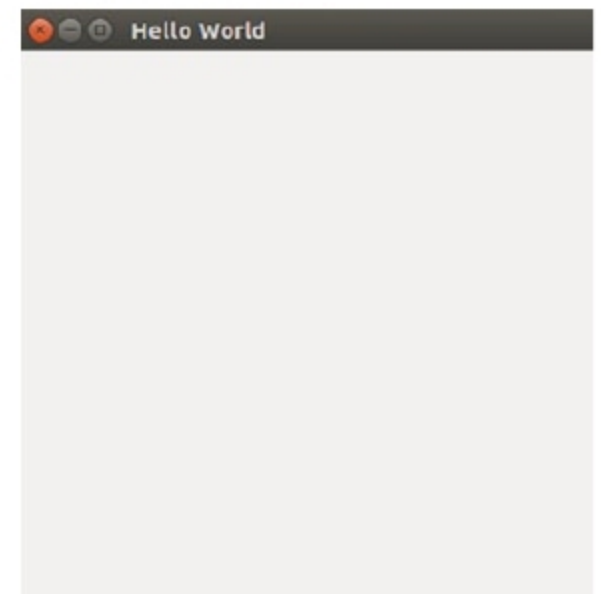


Figure 9: BeeWare 'Hello World' app